

Math4AI 4031 - Final Project

Nazanin Bazgir

K. N. Toosi University of Technology

February 14, 2025

Contents

1	Introduction	4
2	Question 1: Applying SVD to MLP	5
2.1	Dataset	5
2.2	Mathematical Formulation	5
2.3	Implementation of MLP Model	6
2.4	Training Process of MLP Model	7
2.5	Results of MLP Model	8
2.6	Implementation of SVD-based MLP Model	10
2.7	Results of SVD Model	11
2.7.1	Effect on Accuracy	11
2.7.2	Effect on Trainable Parameters	12
2.7.3	Effect on Memory Usage	13
2.7.4	Effect on Training Time	14
2.8	Advantages of SVD in Reducing Parameters	15
3	Question 2: Low-Rank Adaptation (LoRA)	16
3.1	Dataset: Fashion MNIST	16
3.1.1	Dataset Properties	16
3.1.2	Preprocessing Steps	16
3.2	Mathematical Formulation of LoRA	17
3.2.1	Introduction to LoRA	17
3.2.2	LoRA's Low-Rank Approximation	17
3.2.3	Reduction in the Number of Parameters	18
3.2.4	LoRA in Forward Pass	19
3.3	Loading and Training the Pretrained Model	19
3.3.1	Model Training Results	19
3.3.2	Confusion Matrix of the Pretrained Model	19
3.3.3	Learning Curve of the Pretrained Model	20
3.4	Implementation of LoRA	21
3.5	Training Process of LoRA	22
3.6	Evaluation of LoRA Model for $k = 10$	23
3.6.1	Confusion Matrix for $k = 10$	23
3.6.2	Learning Curve for $k = 10$	24
3.7	Comparison of MLP and LoRA Models	24
3.8	plots	25
3.8.1	Analysis of Results	28
4	Question 3: Applying SVD on Transformers	30
4.1	Dataset	30
4.1.1	Loading the Dataset	30
4.1.2	Tokenization Process	30
4.1.3	Applying Tokenization	30
4.1.4	Converting to PyTorch Format	31
4.2	Positional Encoding	31
4.2.1	Mathematical Formulation	31
4.2.2	Implementation in PyTorch	32

4.2.3	Analysis	32
4.3	Multi-Head Self-Attention	32
4.3.1	Mathematical Formulation	32
4.3.2	PyTorch Implementation	33
4.3.3	Analysis	34
4.4	Transformer Block	34
4.4.1	Mathematical Formulation	34
4.4.2	PyTorch Implementation	35
4.4.3	Analysis	35
4.5	Transformermodel	36
4.5.1	PyTorch Implementation	36
4.5.2	Mathematical Formulation	37
4.6	Evaluation of Transformer Model	37
4.6.1	Learning Curve	38
4.6.2	Confusion Matrix	38
4.6.3	Performance Summary	38
4.7	Applying SVD to Self-Attention	39
4.7.1	Mathematical Formulation	39
4.7.2	Reduction in Parameters Using SVD	39
4.7.3	PyTorch Implementation	40
4.7.4	Analysis	42
4.8	Analysis of Plots and Performance Results	42
4.8.1	Performance Summary	42
4.9	Comparison of Standard Transformer and SVD-Optimized Transformer .	43
4.9.1	Performance Comparison	43
4.9.2	Key Observations	44
4.9.3	Analysis of SVD Effectiveness in Transformer Models	44
5	Conclusion	46

Abstract

This report investigates the effectiveness of matrix decomposition techniques in enhancing deep learning model efficiency. We explore the impact of Singular Value Decomposition (SVD) and Low-Rank Adaptation (LoRA) on neural networks, analyzing their effects on performance, computational cost, parameter reduction, and training stability. Through mathematical derivations, dataset preprocessing details, and extensive experimental evaluations, we compare the trade-offs between accuracy and efficiency. Our study demonstrates how SVD optimizations improve model compression and memory efficiency, while LoRA facilitates the optimization of large models, enabling their training with reduced computational cost.

1 Introduction

Deep learning models often require substantial computational resources due to large parameter sets. Decomposition techniques such as Singular Value Decomposition (SVD) and Low-Rank Adaptation (LoRA) provide a way to reduce computational costs while maintaining model accuracy. In this report, we examine the effects of these techniques on different neural network architectures and datasets.

Question 1: Applying SVD to MLP We start by working with the **Hoda Dataset**, a Persian handwritten digit classification dataset. We preprocess the dataset, train a Multi-Layer Perceptron (MLP), and apply SVD to its weight matrices. We analyze the impact of truncating singular values on model accuracy and computational efficiency.

Question 2: Low-Rank Adaptation (LoRA) We experiment with the **Fashion MNIST** dataset, a more complex classification task. We apply LoRA, which introduces two low-rank trainable matrices to approximate weight updates. The goal is to observe improvements in efficiency while maintaining performance.

Question 3: Applying SVD on Transformers For natural language processing, we use the **IMDB sentiment analysis dataset**. We implement a basic transformer model, apply SVD to its attention weight matrices (Query, Key, and Value), and evaluate its efficiency compared to the original transformer architecture.

2 Question 1: Applying SVD to MLP

2.1 Dataset

- **Hoda Dataset:** The Hoda dataset is a well-known dataset for Persian handwritten digit classification. It consists of grayscale images of Persian numerals, widely used for benchmarking machine learning models in Persian Optical Character Recognition (OCR).
- **Dataset Properties:**
 - **Training Set:** 60,000 images
 - **Test Set:** 20,000 images
 - **Image Dimensions:** 32×32 pixels
 - **Number of Classes:** 10 (digits 0 to 9)
 - **Grayscale Images:** Single-channel images with intensity values ranging from 0 to 255
- **Preprocessing Steps:**
 - **Data Loading:** The dataset was loaded from ‘.cdb’ files containing training and test sets.
 - **Reshaping:** Each 32×32 image was reshaped into a vector of size 1024 to be compatible with Multi-Layer Perceptron (MLP) models.
 - **Normalization:** The pixel intensity values were scaled to the range $[0,1]$ by dividing by 255, ensuring stable training dynamics.
 - **Conversion to Tensors:** The dataset was converted into ‘torch.Tensor’ format to facilitate processing in PyTorch.
 - **Label Encoding:** Labels were converted to integer tensors to be compatible with classification models.
 - **Batching:** The dataset was divided into batches to improve computational efficiency during training.

2.2 Mathematical Formulation

To reduce the number of parameters in the MLP model, we apply **Singular Value Decomposition (SVD)** to approximate the weight matrices. Instead of storing and updating the full weight matrix W , we keep only the top k singular values, leading to a more efficient representation.

Standard Weight Representation in MLP in a fully connected layer, the weight matrix W of size $m \times n$ maps an input of dimension m to an output of dimension n . The number of parameters in this case is:

$$P_{\text{full}} = m \times n \tag{1}$$

Using SVD, we decompose W as:

$$W = U \Sigma V^T \tag{2}$$

where:

- $U \in \mathbb{R}^{m \times m}$ is an orthogonal matrix containing left singular vectors.
- $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing singular values.
- $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix containing right singular vectors.

Instead of using the full decomposition, we approximate W by keeping only the top k singular values:

$$W_k = U_{:,1:k} \Sigma_{1:k,1:k} V_{:,1:k}^T \quad (3)$$

This reduces the total number of parameters from:

$$P_{\text{full}} = m \times n \quad (4)$$

to:

$$P_{\text{SVD}} = (m \times k) + (k \times k) + (k \times n) \quad (5)$$

Since $k \ll \min(m, n)$, this technique significantly decreases:

- The number of **trainable parameters**, reducing storage and computational requirements.
- The **memory usage** since the model now requires fewer weights to be stored.
- The **computational cost**, especially in matrix multiplications, since the effective dimensions are now reduced.

Ratio of Parameter Reduction To quantify the reduction, we compute the ratio:

$$\frac{P_{\text{SVD}}}{P_{\text{full}}} = \frac{m \times k + k \times k + k \times n}{m \times n} \quad (6)$$

For small values of k , the parameter reduction is approximately:

$$P_{\text{SVD}} \approx \frac{2k}{m} P_{\text{full}} \quad (7)$$

which clearly shows that using a lower rank k leads to a significant reduction in the number of parameters, especially when $k \ll m$ and $k \ll n$.

2.3 Implementation of MLP Model

The model is defined in PyTorch as follows:

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.flatten = nn.Flatten()

        self.fc1 = nn.Linear(1024, 200)
        self.fc2 = nn.Linear(200, 150)
        self.fc3 = nn.Linear(150, 10)
        self.relu = nn.ReLU()
```

```

def forward(self, x):
    x = self.flatten(x)

    x = self.fc1(x)
    x = self.relu(x)

    x = self.fc2(x)
    x = self.relu(x)

    x = self.fc3(x)

    return x

model = Model().to(device)
print(summary(model, input_size=(1, 1024)))

```

The Multi-Layer Perceptron (MLP) model used for Persian handwritten digit classification is implemented in PyTorch. The model consists of three fully connected layers with ReLU activation functions. Below is the architecture:

- **Input Layer:** The input images of size 32×32 are flattened into a vector of size 1024.
- **Fully Connected Layers:**
 - First layer: $1024 \rightarrow 200$ neurons.
 - Second layer: $200 \rightarrow 150$ neurons.
 - Output layer: $150 \rightarrow 10$ neurons (corresponding to 10 digit classes).
- **Activation Function:** ReLU is used after each hidden layer.
- **Output:** The final layer outputs raw logits for classification into 10 categories.

2.4 Training Process of MLP Model

The training process is implemented as follows:

```

def train_model(model, train_loader, num_epochs=30, lr=0.001,
patience=5):
    device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
    model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    scheduler = ReduceLROnPlateau(optimizer, mode='min', factor
=0.1, patience=patience)

    train_losses = []
    train_accuracies = []

```

```

os.makedirs("./output_MLP", exist_ok=True)

for epoch in range(num_epochs):
    model.train()
    running_train_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        output_MLPs = model(images)
        loss = criterion(output_MLPs, labels)

        _, predicted = torch.max(output_MLPs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_train_loss += loss.item()

    epoch_train_loss = running_train_loss / len(train_loader)
    train_accuracy = 100 * correct / total

    train_losses.append(epoch_train_loss)
    train_accuracies.append(train_accuracy)

    scheduler.step(epoch_train_loss)

```

```

trained_model = train_model(model, train_loader, num_epochs=30,
                             lr=0.001, patience=5)

```

The training process utilizes **CrossEntropyLoss** as the loss function, which is well-suited for multi-class classification tasks. Additionally, the model is optimized using the **Adam Optimizer** with an initial learning rate of 0.001 to efficiently update weights and improve convergence. The final The chosen configuration resulted in an accuracy of **98.28%** on the test set.

2.5 Results of MLP Model

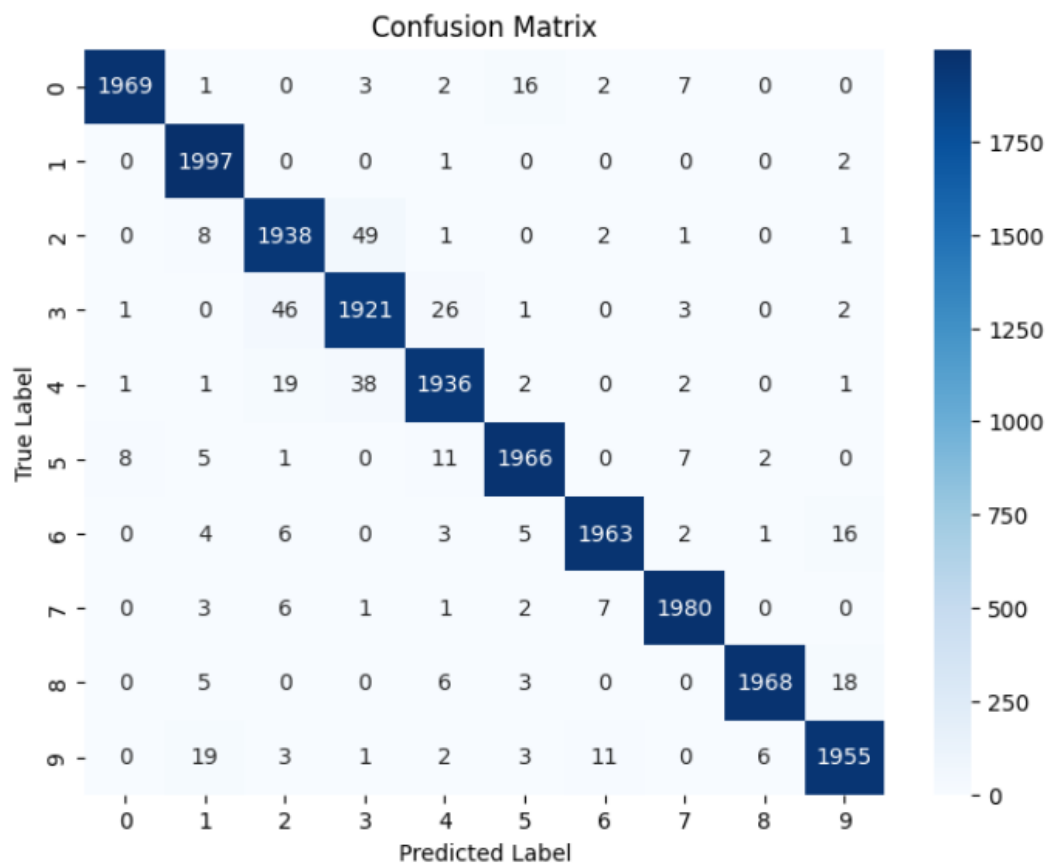


Figure 1: Confusion Matrix of MLP Model

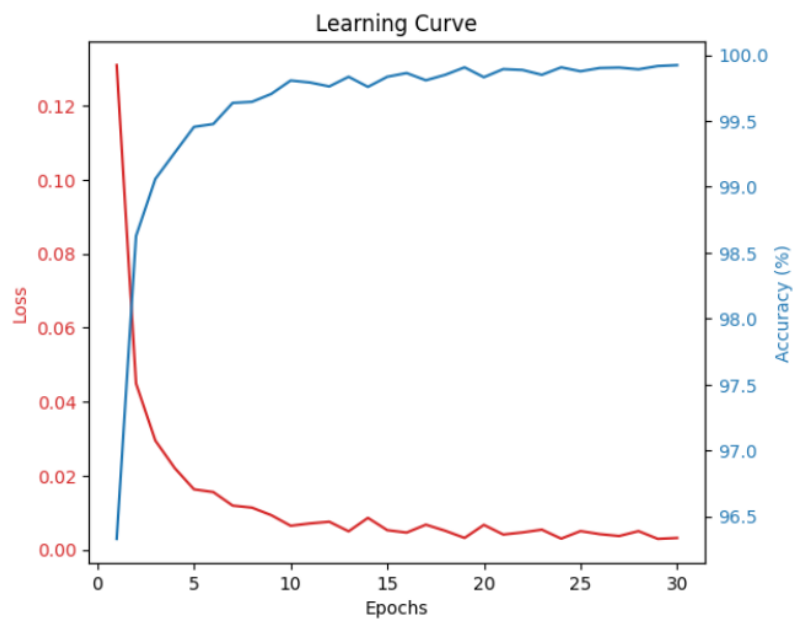


Figure 2: Learning Curve of MLP Model

Layer (type:depth-idx)	Output Shape	Param #
Model	[1, 10]	--
└Flatten: 1-1	[1, 1024]	--
└Linear: 1-2	[1, 200]	205,000
└ReLU: 1-3	[1, 200]	--
└Linear: 1-4	[1, 150]	30,150
└ReLU: 1-5	[1, 150]	--
└Linear: 1-6	[1, 10]	1,510
Total params: 236,660		
Trainable params: 236,660		
Non-trainable params: 0		
Total mult-adds (Units.MEGABYTES): 0.24		
Input size (MB): 0.00		
Forward/backward pass size (MB): 0.00		
Params size (MB): 0.95		
Estimated Total Size (MB): 0.95		

Figure 3: Summary of MLP Model Architecture

2.6 Implementation of SVD-based MLP Model

To reduce the number of parameters in the MLP model while maintaining performance, Singular Value Decomposition (SVD) is applied to decompose the weight matrices of the fully connected layers. Instead of storing large weight matrices, they are approximated using low-rank decomposition.

Key Steps:

- Random initialization of weight matrices for each layer.
- Applying SVD to decompose these weight matrices into lower-dimensional matrices.
- Keeping only the top k singular values to approximate the original weights.
- Using the truncated SVD matrices in place of full-rank weight matrices.

The PyTorch implementation is as follows:

```
class SVD_MLP(nn.Module):
    def __init__(self, k=50):
        super(SVD_MLP, self).__init__()
        self.flatten = nn.Flatten()

        # Define original weight matrices
        W1 = torch.randn(1024, 200) * 0.01
        W2 = torch.randn(200, 150) * 0.01
        W3 = torch.randn(150, 10) * 0.01

        # Perform SVD on the original weight matrices
        U1, S1, V1 = torch.svd(W1)
```

```

U2, S2, V2 = torch.svd(W2)
U3, S3, V3 = torch.svd(W3)

# Truncate SVD to obtain k-rank approximation
self.W1_1 = nn.Parameter(U1[:, :k])
self.W1_2 = nn.Parameter(torch.diag(S1[:k]) @ V1[:, :k].t
    ())

self.W2_1 = nn.Parameter(U2[:, :k])
self.W2_2 = nn.Parameter(torch.diag(S2[:k]) @ V2[:, :k].t
    ())

self.W3_1 = nn.Parameter(U3[:, :k])
self.W3_2 = nn.Parameter(torch.diag(S3[:k]) @ V3[:, :k].t
    ())

self.relu = nn.ReLU()
self.softmax = nn.Softmax(dim=1)

def forward(self, x):
    x = self.flatten(x)

    x = self.relu(x @ self.W1_1 @ self.W1_2)
    x = self.relu(x @ self.W2_1 @ self.W2_2)
    x = self.softmax(x @ self.W3_1 @ self.W3_2)

    return x

```

2.7 Results of SVD Model

To evaluate the performance of the SVD-based MLP model, experiments were conducted with different values of k . The results are summarized in the following figures, including accuracy, memory usage, training time, and parameter efficiency.

2.7.1 Effect on Accuracy

- In the range $k = 10$ to $k = 50$, the model achieves high accuracy, suggesting that a moderate rank provides a balance between efficiency and performance.
- Beyond $k = 50$, accuracy begins to drop. This can be attributed to **overfitting** due to an excessive number of parameters.
- At very low ranks ($k < 5$), accuracy is significantly degraded as the model lacks enough expressive power.

Figure 4 confirms that an optimal k exists within a specific range, and choosing excessively large or small values negatively impacts accuracy.

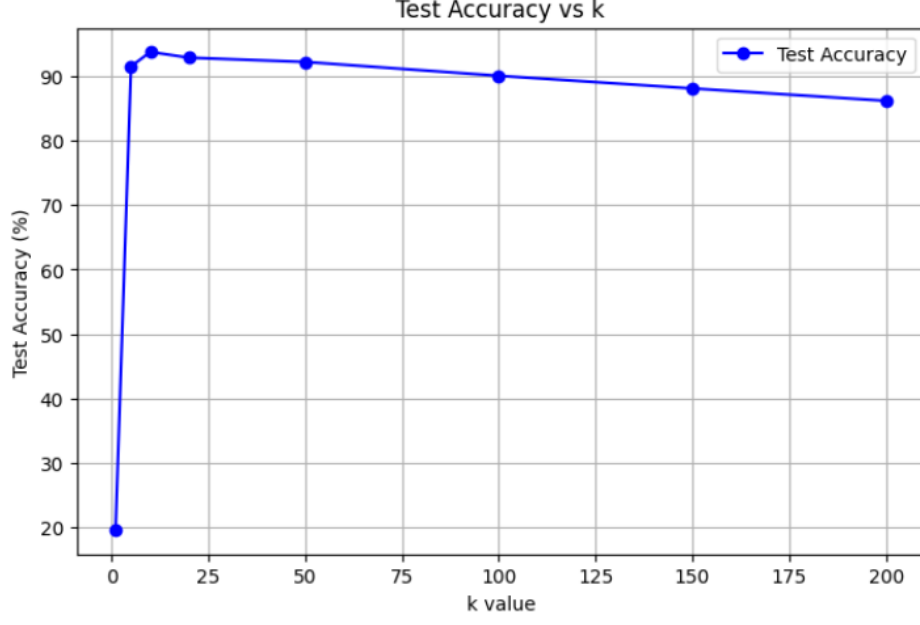


Figure 4: Test Accuracy vs. k

2.7.2 Effect on Trainable Parameters

As k increases, the number of trainable parameters follows a linear growth pattern:

$$P_{\text{LoRA}} = k(m + n) \quad (8)$$

From Figure 5, it is evident that a higher k results in a larger model with increased computational cost.

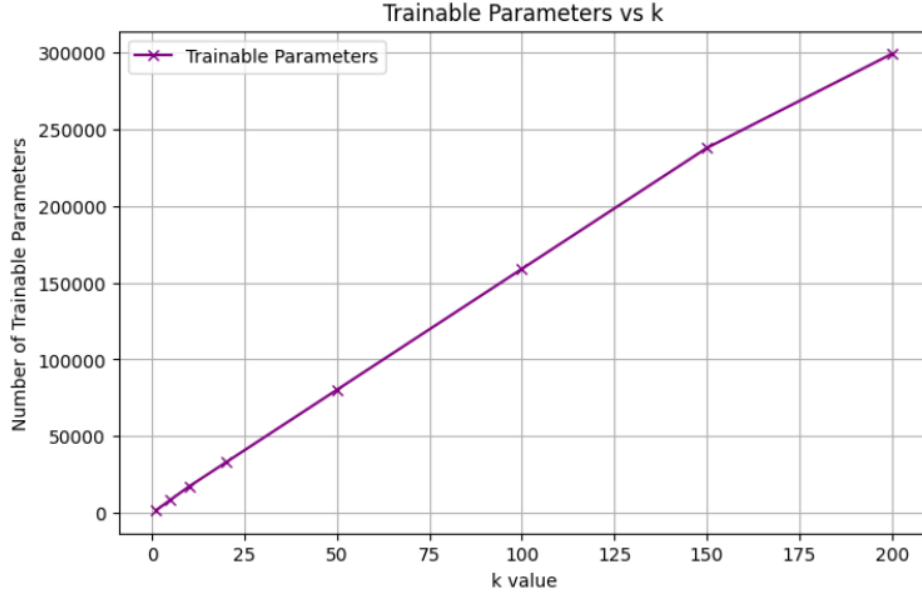


Figure 5: Trainable Parameters vs. k

2.7.3 Effect on Memory Usage

Memory consumption also increases as k grows, as shown in Figure 6. This is due to the increasing size of matrices A and B , which require additional storage.

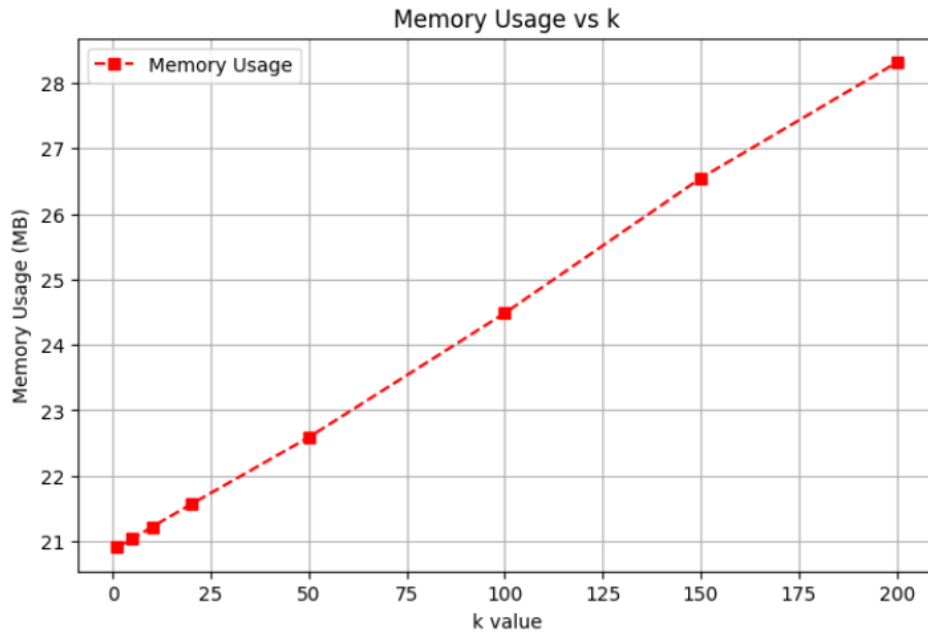


Figure 6: Memory Usage vs. k

2.7.4 Effect on Training Time

Training time is not strictly increasing with k , as seen in Figure 7. While higher ranks require more computation per iteration, larger models might converge faster due to increased capacity, leading to an initial rise and then stabilization in training time.

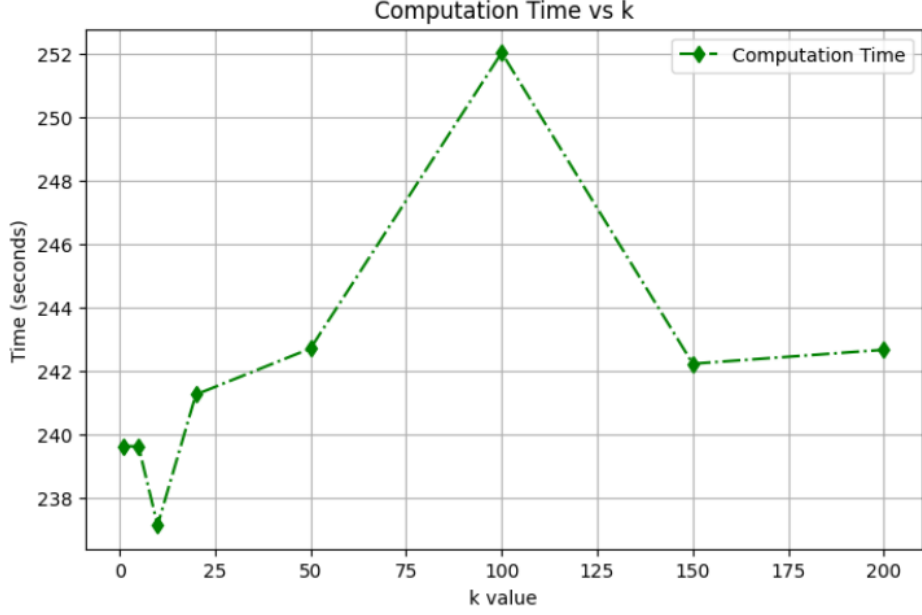


Figure 7: Training Time vs. k

Choosing an optimal k is crucial: too small, and the model lacks expressiveness; too large, and it becomes inefficient with reduced generalization.

k	Test Accuracy (%)	Memory Usage (MB)	Time (sec)	Trainable Params
1	19.535	20.909180	239.645668	1734
5	91.525	21.032227	239.632211	8670
10	93.765	21.216797	237.149863	17340
20	92.870	21.564453	241.262711	33080
50	92.215	22.586426	242.717400	80300
100	90.065	24.484863	252.032645	159000
MLP	98.28	-	-	236,660
150	88.100	26.547852	242.234867	237700
200	86.185	28.314453	242.676356	298900

Table 1: Overall Summary of SVD Model Performance

2.8 Advantages of SVD in Reducing Parameters

The application of Singular Value Decomposition (SVD) in the MLP model has significantly reduced the number of trainable parameters while maintaining a reasonable accuracy.

Comparison with standard MLP model:

- **MLP without SVD:**

- Trainable Parameters: **236,660**
- Test Accuracy: **98.28%**

- **SVD-MLP with $k = 10$:**

- Trainable Parameters: **17,340** (about **92% reduction**)
- Test Accuracy: **93.765%** (only **4.5% drop**)

By evaluating different values of k , we observe that using **low-rank approximations** can significantly reduce model complexity while still maintaining acceptable accuracy. The balance between accuracy, memory usage, and computational cost makes SVD an effective approach for optimizing deep learning models.

Thus, applying SVD to neural networks provides a practical way to optimize model performance, especially in cases where hardware resources are constrained.

3 Question 2: Low-Rank Adaptation (LoRA)

3.1 Dataset: Fashion MNIST

The **Fashion MNIST** dataset is used in this study for training and evaluating the model. It serves as an alternative to the traditional MNIST dataset but contains grayscale images of clothing items instead of handwritten digits. This dataset is widely used for benchmarking deep learning models in image classification tasks.

3.1.1 Dataset Properties

- **Total Images:**
 - Training set: **60,000** images
 - Test set: **10,000** images
- **Image Dimensions:** 32×32 pixels
- **Number of Classes:** 10
 - Categories include: T-shirt, trousers, pullover, dress, coat, sandals, shirt, sneakers, bag, and ankle boots.
- **Color Mode:** Grayscale (Single-channel)

3.1.2 Preprocessing Steps

To ensure the dataset is compatible with the neural network and improves model convergence, the following preprocessing steps are applied:

- **Resizing:** Each image is resized to 32×32 pixels to match the model input dimensions.
- **Tensor Conversion:** The images are converted from PIL format to PyTorch tensors.
- **Normalization:** Pixel values are normalized using:
 - Mean: **0.2860**
 - Standard Deviation: **0.3530**

The dataset is implemented as follows:

```
transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.2860, ), (0.3530, ))
])

# Load Fashion MNIST dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root="./data", train=True, transform=transform, download=True
)
```

```
test_dataset = torchvision.datasets.FashionMNIST(
    root="./data", train=False, transform=transform, download=
    True
)

print(f"Train_dataset_size:{len(train_dataset)}, Test_dataset_
size:{len(test_dataset)}")
```

3.2 Mathematical Formulation of LoRA

3.2.1 Introduction to LoRA

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique designed to improve large-scale deep learning models without modifying all their parameters. Instead of updating the full weight matrices of a pre-trained model, LoRA decomposes the weight update into two smaller trainable matrices, significantly reducing computational cost and memory usage.

In traditional fine-tuning, the weights of a model are updated as:

$$W_{\text{updated}} = W + \Delta W \quad (9)$$

where:

- W is the original pre-trained weight matrix.
- ΔW is the trainable weight update obtained via gradient descent.

However, updating all elements of ΔW requires storing and computing a large number of parameters, making it computationally expensive for large models.

3.2.2 LoRA's Low-Rank Approximation

Instead of learning a full-rank matrix ΔW , LoRA assumes that its updates lie in a lower-dimensional subspace. Thus, the update matrix is decomposed into two smaller matrices A and B :

$$\Delta W \approx AB \quad (10)$$

Thus, the updated weights become:

$$W_{\text{updated}} = W + AB \quad (11)$$

where:

- $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{r \times n}$.
- r is the LoRA rank, a hyperparameter controlling compression ($r \ll m, n$).

Since A and B are low-rank matrices, the number of trainable parameters is greatly reduced compared to learning the full ΔW .

3.2.3 Reduction in the Number of Parameters

In a standard model, the total number of trainable parameters in a weight matrix is given by:

$$P_{\text{full}} = m \times n \quad (12)$$

However, with LoRA, the number of trainable parameters is significantly reduced by factorizing the weight updates into two smaller matrices:

$$P_{\text{LoRA}} = r(m + n) \quad (13)$$

Since r is a hyperparameter chosen such that $r \ll \min(m, n)$, the product $r(m + n)$ is much smaller than $m \times n$. This leads to a significant reduction in the number of parameters, as the ratio between LoRA's parameters and the full model's parameters is:

$$\frac{P_{\text{LoRA}}}{P_{\text{full}}} = \frac{r(m + n)}{m \times n} = \frac{r}{\min(m, n)} \quad (14)$$

Since r is typically orders of magnitude smaller than m and n , the LoRA parameter count is only a small fraction of the original model's trainable parameters. This compression greatly enhances computational efficiency and reduces memory requirements while maintaining model performance.

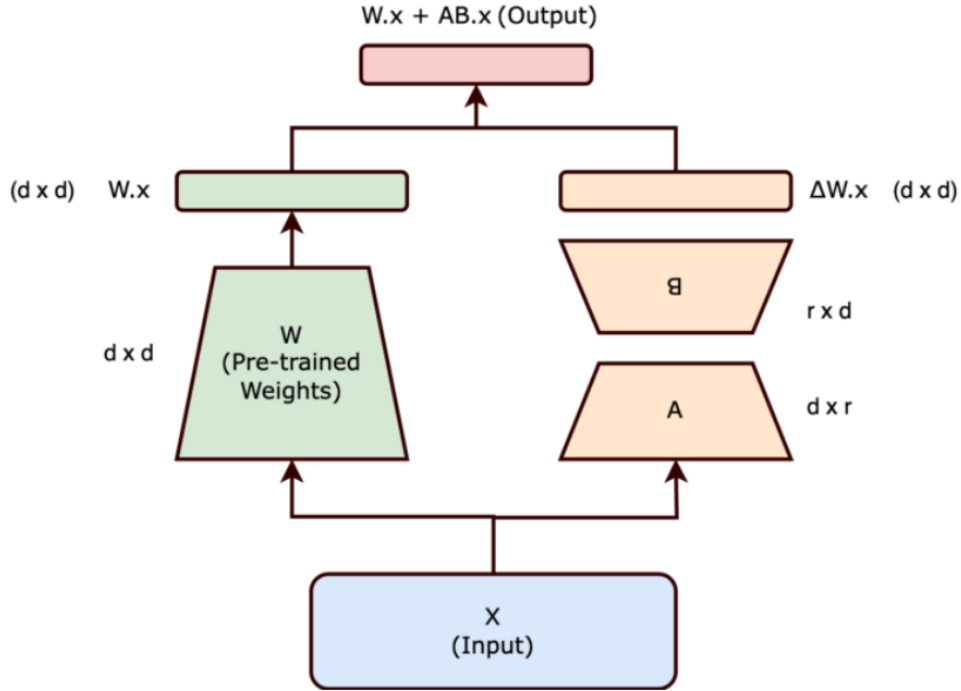


Figure 8: Comparison of weight updates in standard fine-tuning vs. LoRA

For example, if a weight matrix W has a size of 1000×1000 (1 million parameters), in full fine-tuning, all **1 million parameters** must be updated. However, using LoRA with $r = 2$: The dimensions of the low-rank matrices in LoRA for a weight matrix of size 1000×1000 (1 million parameters) are as follows:

- Matrix A consists of 1000 rows and 2 columns, leading to a total of $1000 \times 2 = 2000$ trainable parameters.

- Matrix B has 2 rows and 1000 columns, contributing an additional $2 \times 1000 = 2000$ parameters.

Thus, instead of updating 1,000,000 parameters in full fine-tuning, LoRA only requires training 4000 parameters, achieving a **99.6% reduction in trainable parameters**.

3.2.4 LoRA in Forward Pass

Rather than computing $y = Wx$, the LoRA model computes:

$$y' = (W + AB)x = Wx + ABx \quad (15)$$

This ensures that:

- The pre-trained model weights W remain unchanged.
- Only the low-rank matrices A and B are trained, reducing memory and computation.

3.3 Loading and Training the Pretrained Model

In this experiment, we load the pretrained model from the previous task (Part A) and train it further on the dataset.

Steps Taken:

- Load the pretrained model weights.
- Train the model using the training dataset.
- Evaluate its accuracy, training time, GPU memory usage, and number of trainable parameters.
- Analyze the classification results using a confusion matrix and learning curve.

3.3.1 Model Training Results

After training the loaded model, we obtained the following performance metrics:

Model	Accuracy (%)	Training Time (s)	GPU Memory (MB)	Trainable Params
MLP Trained	88.75	310.837	18.9629	236660

Table 2: Performance metrics of the pretrained model after fine-tuning

3.3.2 Confusion Matrix of the Pretrained Model

.

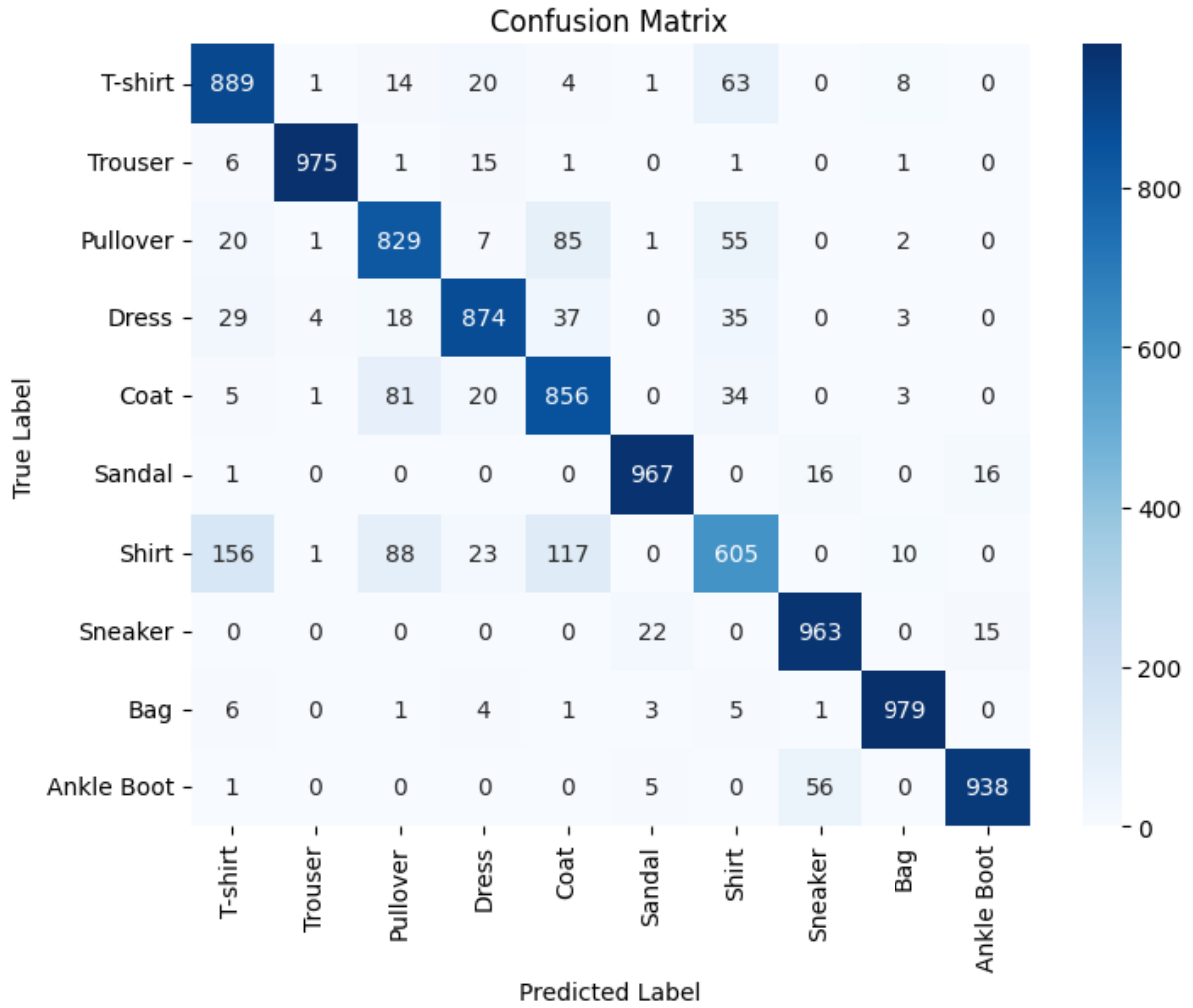


Figure 9: Confusion Matrix of the Pretrained Model

3.3.3 Learning Curve of the Pretrained Model

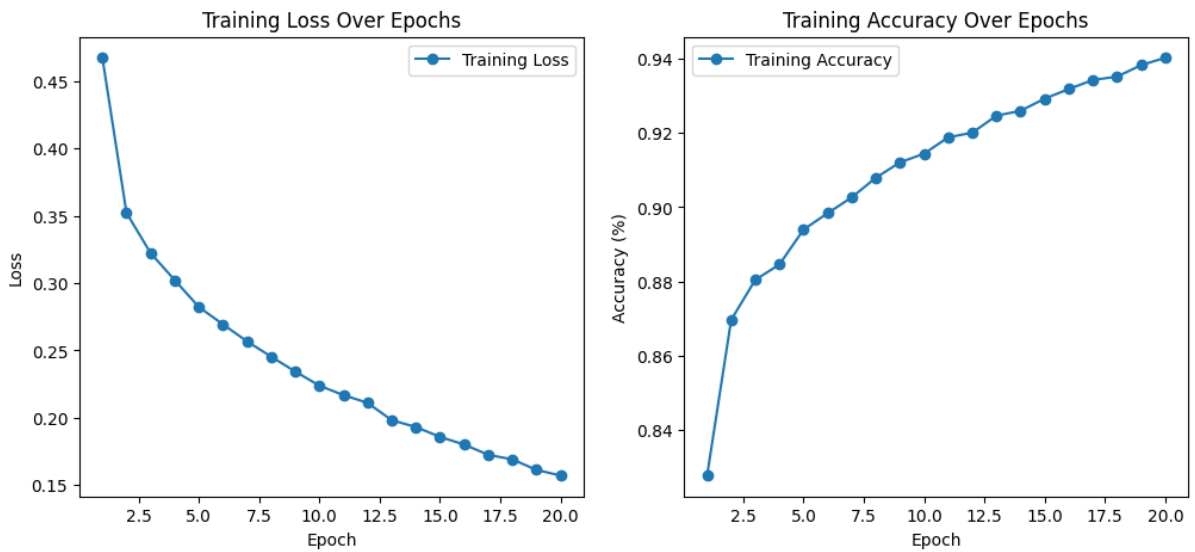


Figure 10: Learning Curve of the Pretrained Model

Observations:

- The model achieves an accuracy of **88.75%**, which is a strong performance.
- The confusion matrix shows well-distributed correct classifications across different classes.
- The learning curve indicates stable convergence, suggesting that the model has effectively learned the dataset.

3.4 Implementation of LoRA

To integrate Low-Rank Adaptation (LoRA) into the MLP model, we define a custom LoRA module that replaces standard fully connected layers. Instead of training the full weight matrix, LoRA decomposes it into two smaller trainable matrices, significantly reducing the number of parameters.

Key Components of LoRA Implementation:

- The LoRA class wraps an existing layer and replaces its weight updates with two smaller matrices, A and B .
- The `apply_lora_to_model` function applies LoRA modifications to all fully connected layers in the model.
- The rank r determines the compression level and efficiency gain.

The LoRA module is implemented as follows:

```
class LoRA(nn.Module):
    def __init__(self, original_layer, rank):
        super(LoRA, self).__init__()
        self.original_layer = original_layer
        self.rank = rank

        # LoRA trainable weights
        self.lora_A = nn.Linear(original_layer.in_features, rank,
                                bias=False)
        self.lora_B = nn.Linear(rank, original_layer.out_features,
                                bias=False)

        # Initialize weights
        nn.init.normal_(self.lora_A.weight, mean=0.0, std=0.01)
        nn.init.zeros_(self.lora_B.weight)

    def forward(self, x):
        return self.original_layer(x) + self.lora_B(self.lora_A(x))
```

Applying LoRA to the MLP Model:

To integrate LoRA into the existing MLP architecture, we replace the fully connected layers with their LoRA-enhanced versions:

```
def apply_lora_to_model(model, rank):
    model.fc1 = LoRA(model.fc1, rank)
    model.fc2 = LoRA(model.fc2, rank)
    model.fc3 = LoRA(model.fc3, rank)
    return model
```

Explanation:

- The function `apply_lora_to_model` replaces the dense layers in the model with LoRA-enhanced layers.
- The original layers remain functional while the LoRA layers introduce trainable low-rank updates.
- The model retains its original expressive power but benefits from reduced trainable parameters and lower computational cost.

3.5 Training Process of LoRA

The training process is implemented as follows:

```
# Different Rank values for LoRA
rank_values = [1, 5, 10, 15, 20, 25, 30, 40, 50, 100, 150]

for rank in rank_values:
    print(f"\nTraining LoRA Model with Rank={rank}")

    # Load the model and apply LoRA
    model_lora = torch.load(model_path, map_location=device)

    for param in model_lora.parameters():
        param.requires_grad = False

    model_lora = apply_lora_to_model(model_lora, rank)
    model_lora.to(device)

    # Train LoRA model
    training_time_lora, train_losses_lora, train_accuracies_lora
        = train_model(model_lora, train_loader, num_epochs=20)

    accuracy_lora = evaluate_model(model_lora, test_loader)
    memory_allocated_lora, memory_reserved_lora = measure_memory
        ()
    trainable_params_lora = count_trainable_params(model_lora)

    results.append({
        "Model": f"LoRA_Rank_{rank}",
        "Accuracy(%)": accuracy_lora,
        "TrainingTime(s)": training_time_lora,
        "GPU_Memory_Used(MB)": memory_allocated_lora,
        "Trainable_Params": trainable_params_lora
    })
```

```

class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle_Boot"]
plot_confusion_matrix(model_lora, test_loader, class_names)
plot_learning_curves(train_losses_lora, train_accuracies_lora, num_epochs=20)

```

3.6 Evaluation of LoRA Model for $k = 10$

After defining and implementing the LoRA-enhanced model, we analyze its performance for a specific rank value. As discussed earlier, the choice of k significantly affects accuracy, memory efficiency, and computational cost. In this section, we present the evaluation results for $k = 10$ as an example.

3.6.1 Confusion Matrix for $k = 10$

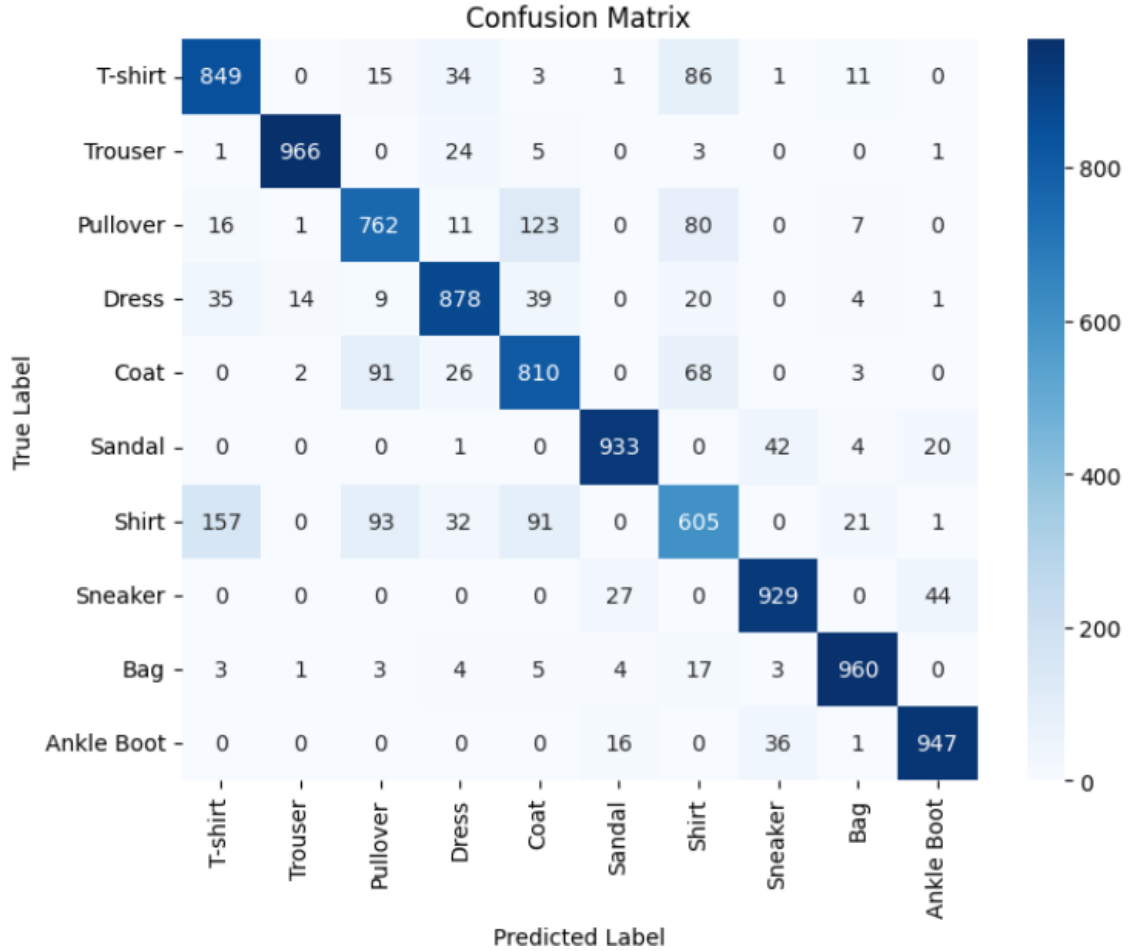


Figure 11: Confusion Matrix for LoRA Model with $k = 10$

3.6.2 Learning Curve for $k = 10$

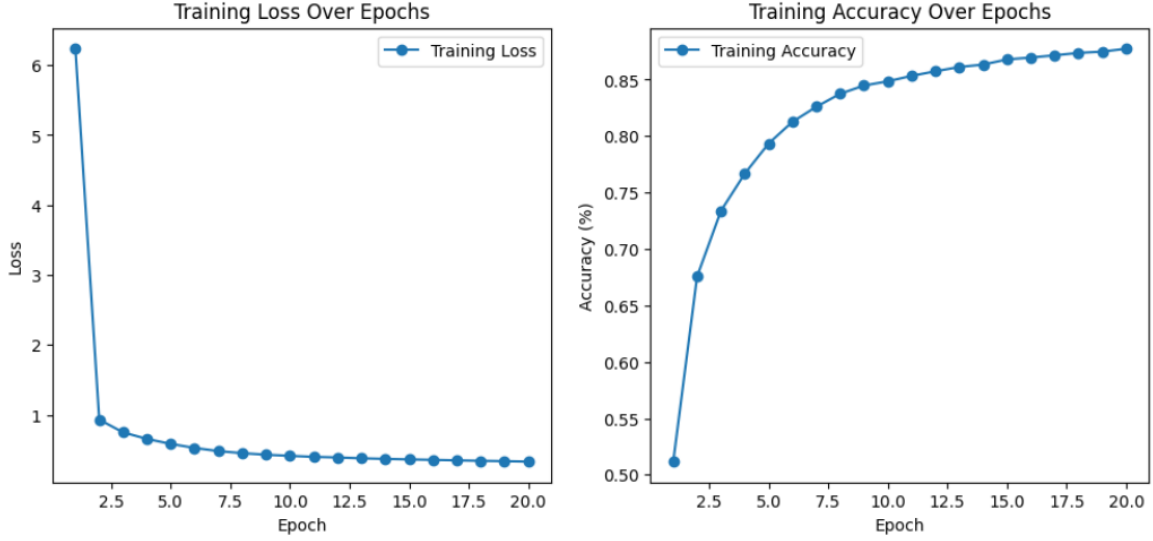


Figure 12: Learning Curve for LoRA Model with $k = 10$

Observations:

- The confusion matrix indicates a well-balanced classification with a reasonable number of correct predictions.
- These results validate the effectiveness of using LoRA with a moderate rank ($k = 10$), striking a balance between accuracy and computational efficiency.

3.7 Comparison of MLP and LoRA Models

To evaluate the effectiveness of LoRA in comparison to a fully trained MLP model, we conducted multiple experiments using different values of LoRA rank k . The table below presents the classification accuracy, training time, GPU memory usage, and number of trainable parameters for both the original MLP and various LoRA configurations.

Model	Accuracy (%)	Training Time (s)	GPU Memory (MB)	Trainable Params
MLP Trained	88.75	310.837	18.9629	236660
LoRA Rank 1	75.74	314.284	19.8838	1734
LoRA Rank 5	84.73	314.700	19.9346	8670
LoRA Rank 10	86.39	314.727	20.001	17340
LoRA Rank 15	86.30	313.066	20.0684	26010
LoRA Rank 20	87.36	315.559	20.1348	34680
LoRA Rank 25	87.08	309.922	20.2012	43350
LoRA Rank 30	87.00	311.226	20.2666	52020
LoRA Rank 40	86.28	313.161	20.3984	69360
LoRA Rank 50	86.88	312.441	20.5312	86700
LoRA Rank 100	87.66	311.486	21.1934	173400
LoRA Rank 150	87.53	312.572	21.8535	260100

Table 3: Comparison of MLP and LoRA models across different ranks

3.8 plots

To further analyze the impact of different LoRA ranks, we present four key performance metrics:

- **Accuracy vs. Model:** Shows how different LoRA ranks affect classification accuracy.

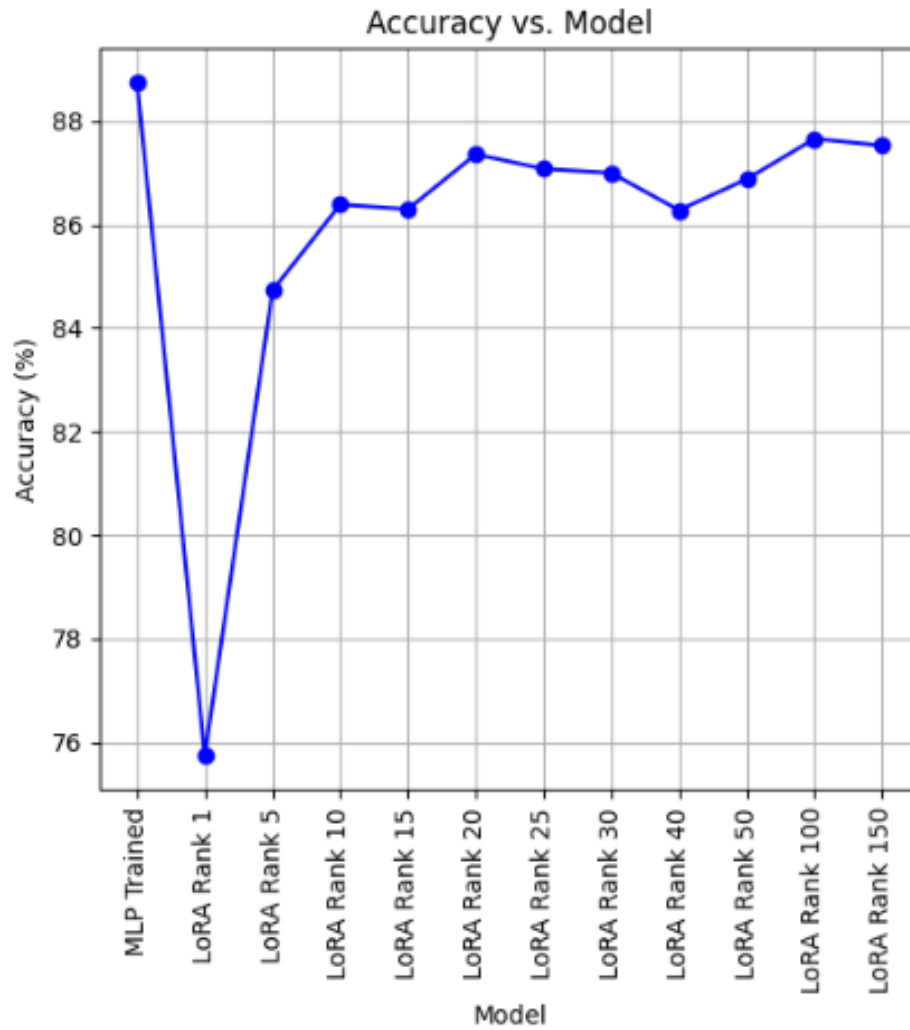


Figure 13: Accuracy vs. Model

- **Training Time vs. Model:** Illustrates the computational cost in terms of training duration.

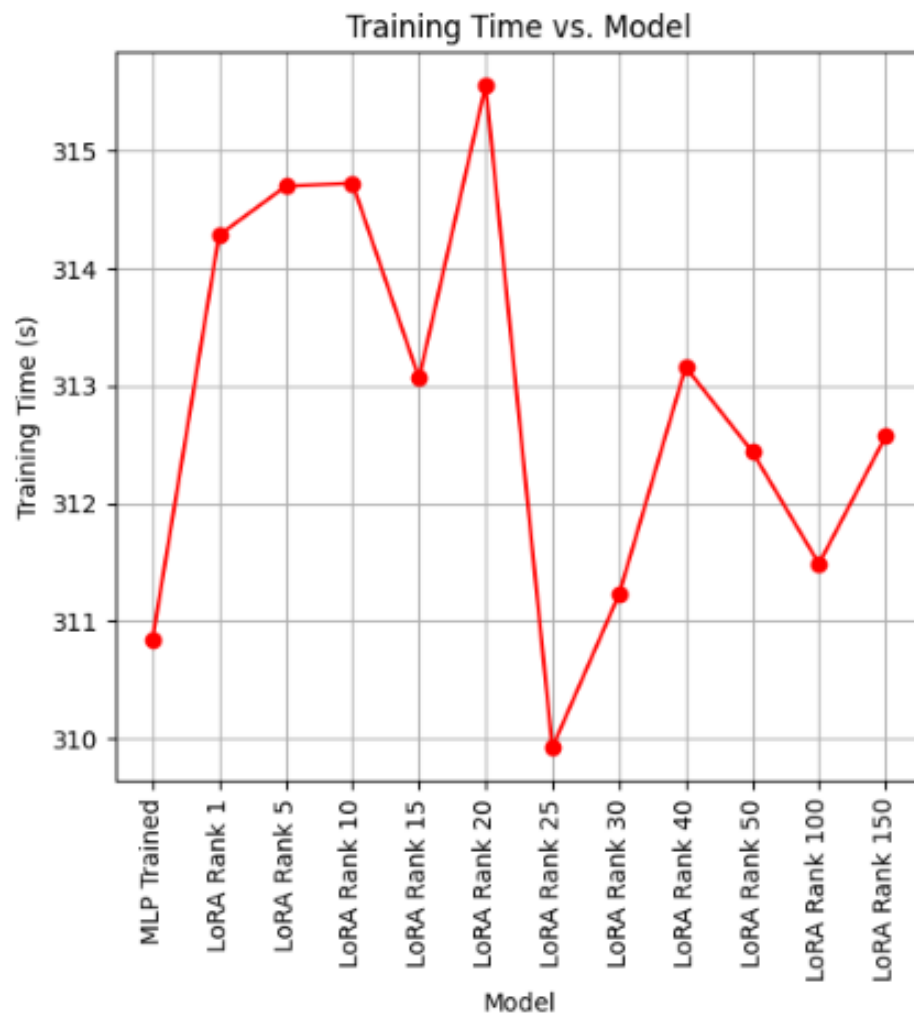


Figure 14: Training Time vs. Model

- **GPU Memory Usage vs. Model:** Represents how GPU memory consumption changes with varying ranks.

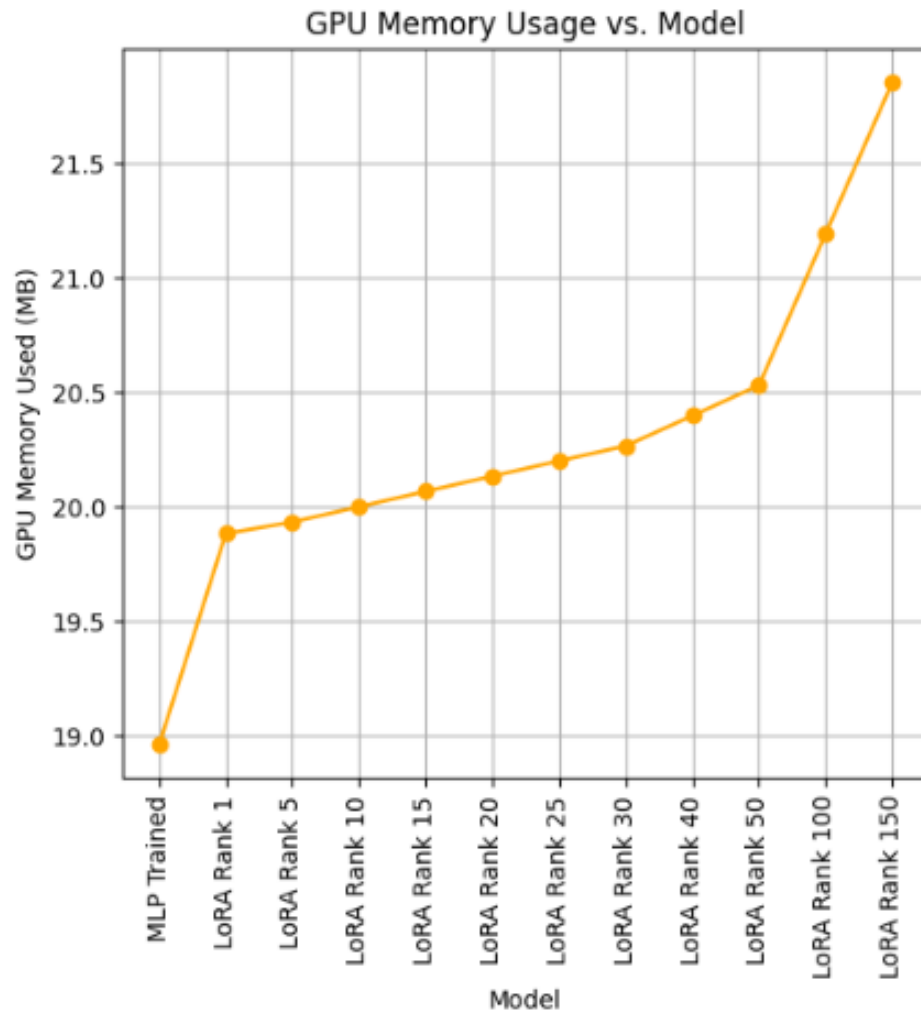


Figure 15: Memory Usage vs. Model

- **Trainable Parameters vs. Model:** Demonstrates the reduction in trainable parameters compared to the fully trained MLP.

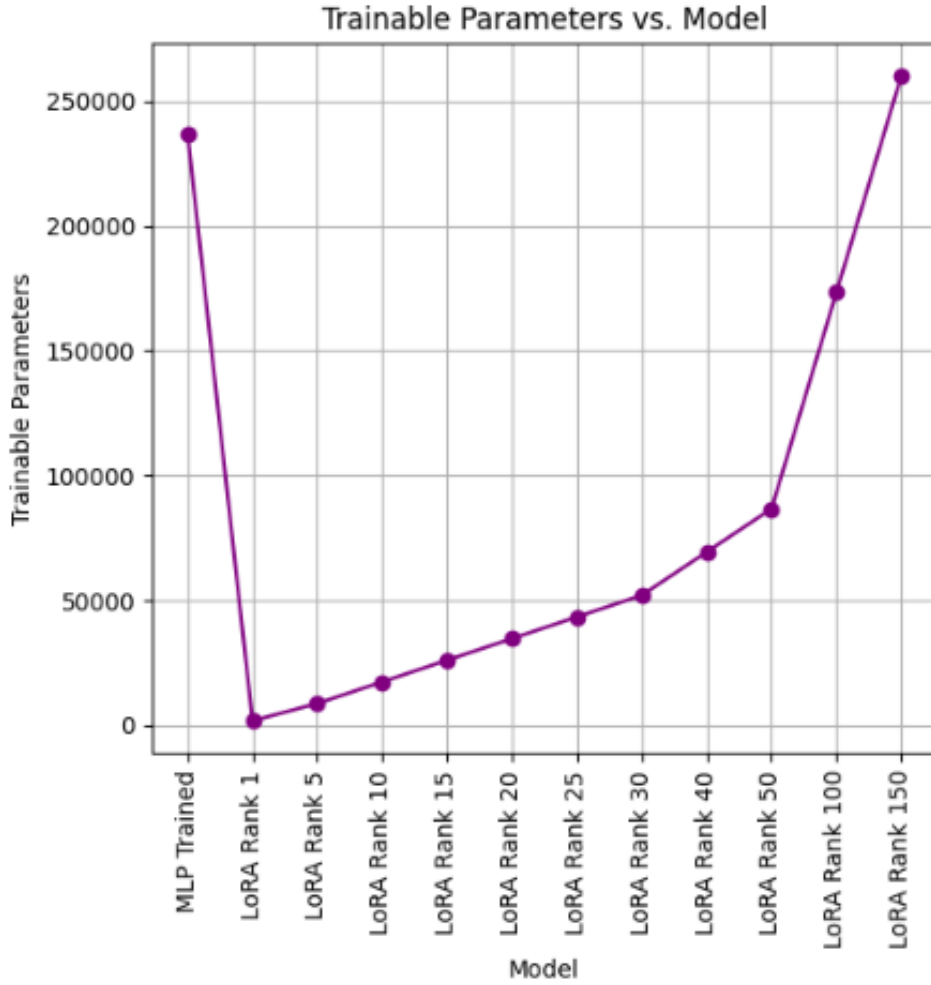


Figure 16: Trainable Parameters vs. Model

3.8.1 Analysis of Results

The following key insights can be drawn from the comparison:

- **Accuracy:** The fully trained MLP model achieves the highest accuracy of 88.75%. However, LoRA with appropriate rank values (e.g., $k = 20$, $k = 50$, and $k = 100$) achieves nearly similar accuracy (87.36% to 87.66%), showing that LoRA can preserve model performance while reducing the number of trainable parameters.

- **Training Time:**

Previous analysis suggested that LoRA does not significantly impact training time across different rank configurations. However, upon further investigation, multiple factors influence the training time dynamics of LoRA:

Factors that can speed up LoRA training:

- **Better initialization:** Since LoRA starts from a pretrained model, it does not require random initialization, leading to faster convergence.

- **Fewer trainable parameters:** The number of trainable parameters in LoRA is significantly lower than a full MLP, which can reduce computational overhead.

Factors that can slow down LoRA training:

- **Additional matrix multiplications:** In each forward pass, LoRA introduces additional matrix multiplications ($W' = W + AB$), increasing computational cost.
- **Increased number of epochs required:** LoRA models often require more epochs to reach the same accuracy as the full model.
- **Domain shift between datasets:** If the source dataset (Hoda) differs significantly from the target dataset (Fashion MNIST), the transferred knowledge may not be as useful, leading to longer convergence times.
- **Matrix approximations:** The low-rank decomposition introduces approximations that might require additional training to compensate for information loss.

Conclusion: The overall impact of LoRA on training time depends on the balance between reduced parameter count and increased computational cost. If the computational savings from fewer parameters outweigh the additional matrix operations, training can be faster. Otherwise, LoRA might require more time to converge, especially when transferring across different datasets.

However, for large-scale models, LoRA provides a significant advantage by reducing the number of trainable parameters, making fine-tuning computationally feasible while maintaining high performance.

- **Memory Usage:** As expected, GPU memory consumption increases as k grows, with LoRA Rank 150 consuming about 21.85MB, compared to 18.96MB for the original MLP.
- **Trainable Parameters:** The most significant benefit of LoRA is the drastic reduction in trainable parameters. The MLP model requires 236,660 parameters, whereas LoRA Rank 1 reduces this to only 1,734 parameters, and even LoRA Rank 50 uses only 86,700 parameters, which is nearly three times smaller than the full model.
- **Trade-Off Between Efficiency and Accuracy:** The best trade-off occurs between $k = 10$ to $k = 50$, where the accuracy remains above 86%, while trainable parameters are significantly reduced compared to the full model.

Conclusion: LoRA provides a memory-efficient alternative to full fine-tuning while maintaining comparable accuracy. For most applications, selecting an optimal k value (e.g., $10 \leq k \leq 50$) can strike a balance between performance, computational efficiency, and memory usage.

4 Question 3: Applying SVD on Transformers

4.1 Dataset

For this task, we use the **IMDB dataset**, a widely recognized benchmark for sentiment analysis. The dataset consists of movie reviews labeled as either *positive* or *negative*, making it suitable for binary classification tasks.

Dataset Properties:

- **Total Samples:** 50,000 movie reviews.
- **Training Set:** 25,000 labeled reviews.
- **Test Set:** 25,000 labeled reviews.
- **Classes:** Positive (1) and Negative (0).

4.1.1 Loading the Dataset

The dataset is loaded using the Hugging Face **datasets** library, which provides an efficient way to manage large text datasets.

```
dataset = load_dataset("imdb")
train_data = dataset["train"]
test_data = dataset["test"]

print(f"Number of training samples: {len(train_data)}")
print(f"Number of test samples: {len(test_data)}")
```

After loading, the dataset is divided into **training** and **test** sets, each containing 25,000 samples.

4.1.2 Tokenization Process

Since deep learning models, especially Transformers, require numerical input, we tokenize the dataset using the **BERT tokenizer**. Tokenization converts raw text into numerical representations that the model can process. We use the **pretrained "bert-base-uncased" tokenizer** with a fixed maximum sequence length.

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
max_seq_length = 256
```

4.1.3 Applying Tokenization

Each text review is tokenized by breaking it into subwords and mapping them to their corresponding BERT vocabulary indices. Padding and truncation are applied to ensure uniform input size.

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
```

```

        max_length=max_seq_length,
        return_tensors="pt"
    )

```

4.1.4 Converting to PyTorch Format

To efficiently train our model, we convert the tokenized dataset into PyTorch tensors, ensuring compatibility with deep learning frameworks.

```

columns = ["input_ids", "attention_mask", "label"]
train_data.set_format(type="torch", columns=columns)
test_data.set_format(type="torch", columns=columns)

```

4.2 Positional Encoding

Since Transformer models do not inherently capture the order of sequences, we use **Positional Encoding** to introduce position information into input embeddings. The encoding is computed using sinusoidal functions, ensuring that each position has a unique representation while maintaining distance-aware properties.

4.2.1 Mathematical Formulation

The positional encoding is computed as follows:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (16)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (17)$$

where:

- pos represents the position of the word in the sequence.
- i is the dimension index.
- d is the embedding dimension.

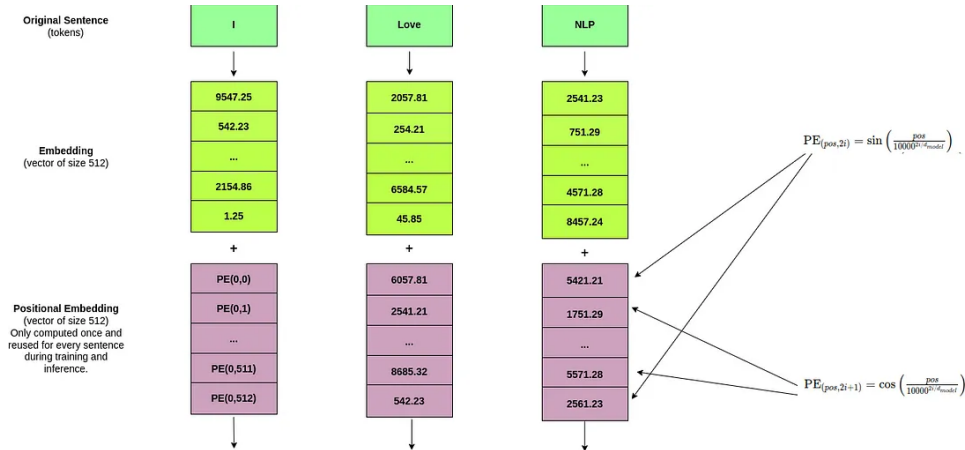


Figure 17: Positional Encoding and embedding

4.2.2 Implementation in PyTorch

The following PyTorch implementation constructs the sinusoidal positional encodings and adds them to the input embeddings.

```
class PositionalEncoding(nn.Module):
    def __init__(self, embed_dim, max_seq_length):
        super(PositionalEncoding, self).__init__()

        pos = torch.arange(0, max_seq_length).unsqueeze(1).float()
        div_term = torch.exp(torch.arange(0, embed_dim, 2).float()
                               () * (-math.log(10000.0) / embed_dim))

        # Compute sinusoidal positional encodings
        pe = torch.zeros(max_seq_length, embed_dim)
        pe[:, 0::2] = torch.sin(pos * div_term)
        pe[:, 1::2] = torch.cos(pos * div_term)

        self.register_buffer("positional_encoding", pe.unsqueeze(0))

    def forward(self, x):
        return x + self.positional_encoding[:, :x.shape[1], :]
```

4.2.3 Analysis

This encoding scheme allows the model to differentiate words based on their position while ensuring that nearby words have similar encodings, helping the model learn positional dependencies effectively.

4.3 Multi-Head Self-Attention

Self-Attention is a crucial mechanism in transformer models that enables the network to capture dependencies between words in a sequence, regardless of their distance. To enhance learning capacity, we use **Multi-Head Self-Attention (MHSA)**, where the attention mechanism is applied in parallel across multiple heads.

4.3.1 Mathematical Formulation

For each attention head, the attention scores are computed using the scaled dot-product mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (18)$$

where:

- $Q = XW_Q$ (Query Matrix)
- $K = XW_K$ (Key Matrix)

- $V = XW_V$ (Value Matrix)
- d_k is the dimensionality of each attention head.

After computing attention outputs for all heads, they are concatenated and passed through a linear transformation:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (19)$$

4.3.2 PyTorch Implementation

The following PyTorch implementation efficiently computes multi-head self-attention.

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_dim, num_heads):
        super(MultiHeadSelfAttention, self).__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        # Initialize weights
        self.W_q = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_k = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_v = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_o = nn.Linear(embed_dim, embed_dim, bias=False)

    def forward(self, x, mask=None):
        batch_size, seq_length, embed_dim = x.shape

        Q = self.W_q(x)
        K = self.W_k(x)
        V = self.W_v(x)

        # Reshape to multiple heads
        Q = Q.view(batch_size, seq_length, self.num_heads, self.head_dim).transpose(1, 2)
        K = K.view(batch_size, seq_length, self.num_heads, self.head_dim).transpose(1, 2)
        V = V.view(batch_size, seq_length, self.num_heads, self.head_dim).transpose(1, 2)

        # Compute attention scores
        scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(self.head_dim, dtype=torch.float32))

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))

        # Compute attention weights
        attn_weights = F.softmax(scores, dim=-1)

        attn_output = torch.matmul(attn_weights, V)
```

```

# Reshape back to original dimensions
attn_output = attn_output.transpose(1, 2).contiguous().
    view(batch_size, seq_length, embed_dim)

output = self.W_o(attn_output)
return output

```

4.3.3 Analysis

- **Multiple Attention Heads:** The attention mechanism is applied independently across multiple subspaces.
- **Dimensionality Reduction:** Each head operates on a reduced dimension d_k to keep computations efficient.
- **Enhanced Representations:** By combining multiple attention heads, the model can learn a richer representation of the input sequence.

This mechanism allows the Transformer to effectively capture both local and global dependencies in sequential data.

4.4 Transformer Block

The Transformer model consists of stacked layers, where each layer is composed of two essential components:

- **Multi-Head Self-Attention:** Captures dependencies between words in a sequence, irrespective of their position.
- **Feed-Forward Network (FFN):** Applies non-linearity and projects the embeddings to a higher-dimensional space.

To ensure stability during training, each sub-layer is followed by:

- **Layer Normalization:** Helps in stabilizing activations and gradients.
- **Dropout:** Prevents overfitting by randomly deactivating neurons.

4.4.1 Mathematical Formulation

Each Transformer block processes input embeddings X as follows:

$$\text{Attention Output} = \text{MultiHead}(X, X, X) \quad (20)$$

$$\tilde{X} = \text{LayerNorm}(X + \text{Dropout}(\text{Attention Output})) \quad (21)$$

$$\text{FFN Output} = \text{ReLU}(W_1 \tilde{X} + b_1) W_2 + b_2 \quad (22)$$

$$Y = \text{LayerNorm}(\tilde{X} + \text{Dropout}(\text{FFN Output})) \quad (23)$$

where:

- W_1, W_2 are learnable weight matrices in the feed-forward network.
- b_1, b_2 are biases.
- ReLU is the activation function.

4.4.2 PyTorch Implementation

The following PyTorch code implements a single Transformer block.

```
class TransformerBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, ff_dim, dropout=0.1):
        :
        super(TransformerBlock, self).__init__()
        self.attention = MultiHeadSelfAttention(embed_dim,
            num_heads)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.norm2 = nn.LayerNorm(embed_dim)

        self.feed_forward = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.ReLU(),
            nn.Linear(ff_dim, embed_dim),
        )

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        attn_output = self.attention(x)
        x = self.norm1(x + self.dropout(attn_output))

        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))

        return x
```

4.4.3 Analysis

- **Self-Attention Mechanism:** Allows the model to capture long-range dependencies in the input sequence.
- **Residual Connections:** Improve gradient flow, making deep transformers trainable.
- **Normalization and Dropout:** Ensure stability and prevent overfitting.

This Transformer block serves as a fundamental building block in deep learning architectures such as BERT and GPT models.

4.5 Transformer model

4.5.1 PyTorch Implementation

```
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_heads, ff_dim,
                  num_layers, num_classes, max_seq_length, dropout=0.1):
        """
        Transformer-based text classification model.
        """
        super(TransformerClassifier, self).__init__()

        # Token embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # Positional encoding
        self.positional_encoding = PositionalEncoding(embed_dim,
                                                       max_seq_length)

        # Transformer blocks
        self.transformer_blocks = nn.ModuleList(
            [TransformerBlock(embed_dim, num_heads, ff_dim,
                              dropout) for _ in range(num_layers)]
        )

        # Fully connected output layer
        self.fc = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        """
        Forward pass of the Transformer Classifier.
        """
        batch_size, seq_length = x.shape

        # Token + Positional encoding
        x = self.embedding(x)
        x = self.positional_encoding(x)

        # Passing through Transformer blocks
        for transformer in self.transformer_blocks:
            x = transformer(x)

        # Global average pooling over sequence length
        x = x.mean(dim=1)

        # Final classification layer
        output = self.fc(x)

        return output
```

The **Transformer-based classifier** is designed for text classification tasks. It consists of the following main components:

- **Embedding Layer:** Converts input tokens into dense vectors.
- **Positional Encoding:** Adds positional information to embeddings.
- **Stacked Transformer Blocks:** A sequence of self-attention and feed-forward layers.
- **Global Average Pooling:** Reduces the sequence dimension.
- **Fully Connected Output Layer:** Produces the final class predictions.

4.5.2 Mathematical Formulation

Let $X \in \mathbb{R}^{B \times T}$ be the input batch of sequences with:

- B as the batch size
- T as the sequence length

The model processes input as follows:

$$E = \text{Embedding}(X) + \text{PositionalEncoding}(X) \quad (24)$$

Each Transformer block updates the representation:

$$H_l = \text{TransformerBlock}(H_{l-1}) \quad (25)$$

After all Transformer layers, we apply mean pooling:

$$Z = \frac{1}{T} \sum_{t=1}^T H_L \quad (26)$$

Finally, the classification output is obtained using:

$$Y = \text{softmax}(ZW + b) \quad (27)$$

4.6 Evaluation of Transformer Model

The performance of the Transformer model is evaluated based on key metrics, including its learning curve, confusion matrix, and overall accuracy, training time, and number of parameters.

4.6.1 Learning Curve

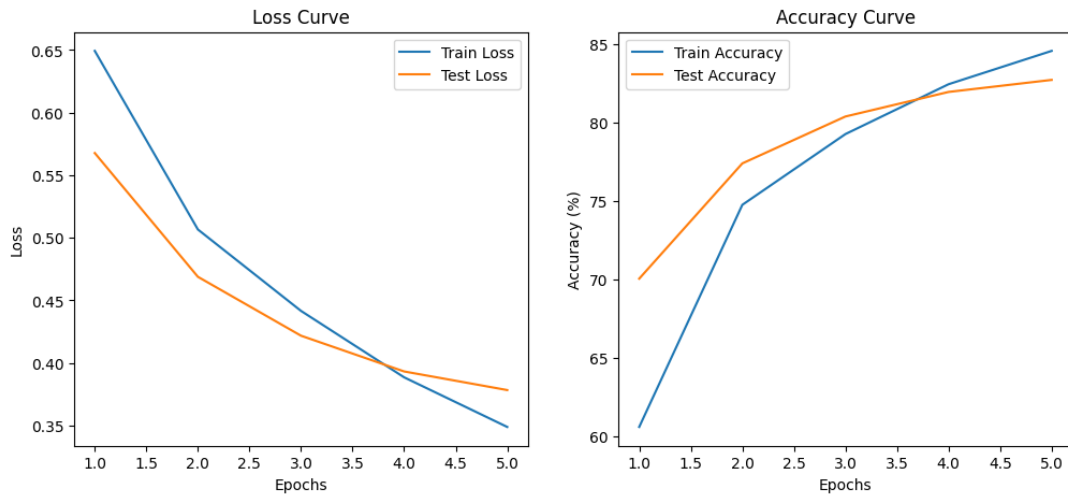


Figure 18: Learning Curve of Transformer Model

4.6.2 Confusion Matrix

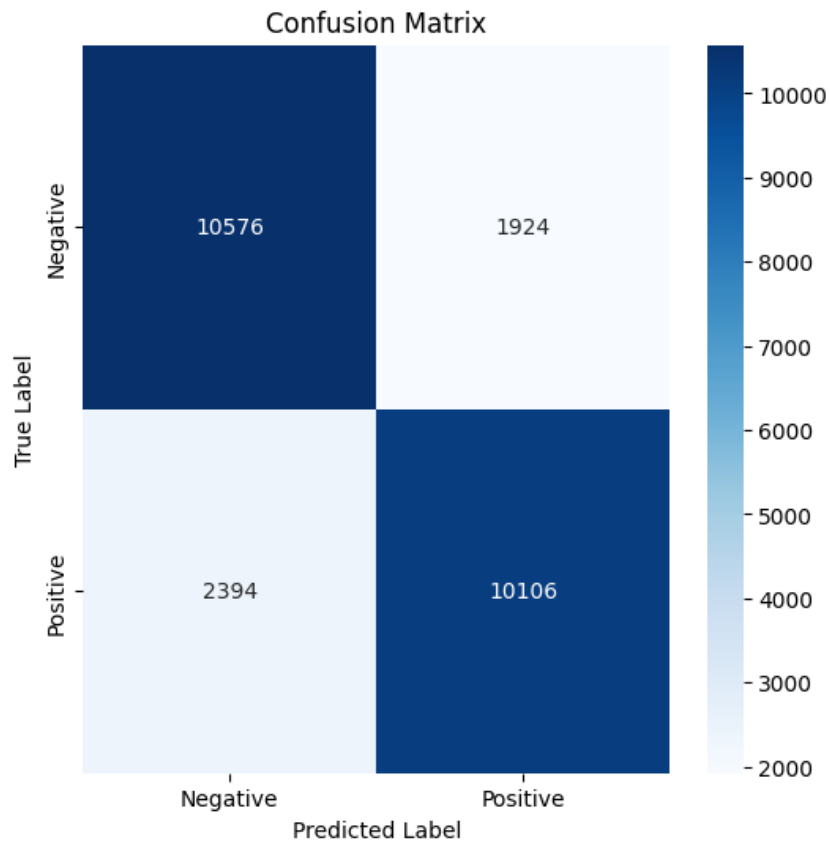


Figure 19: Confusion Matrix of Transformer Model

4.6.3 Performance Summary

Table 4 summarizes the main performance metrics of the Transformer model.

Metric	Value
Test Accuracy (%)	82.73
Training Time (seconds)	112.20
Number of Parameters	4,302,594

Table 4: Performance Summary of Transformer Model

4.7 Applying SVD to Self-Attention

To optimize the computational efficiency of the self-attention mechanism in Transformers, we apply **Singular Value Decomposition (SVD)** to reduce the rank of the weight matrices used for computing queries, keys, and values.

4.7.1 Mathematical Formulation

The standard self-attention mechanism involves three weight matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (28)$$

To reduce the number of trainable parameters, we approximate these matrices using truncated SVD:

$$W_Q \approx U_Q S_Q V_Q^T, \quad W_K \approx U_K S_K V_K^T, \quad W_V \approx U_V S_V V_V^T \quad (29)$$

where:

- U and V are the left and right singular vectors.
- S is the diagonal matrix of singular values.
- The rank of approximation is reduced to a predefined lowRank.

By using truncated SVD, we significantly reduce the number of parameters while maintaining the representational power of the model.

4.7.2 Reduction in Parameters Using SVD

By applying Singular Value Decomposition (SVD) to the weight matrices of the self-attention mechanism, we significantly reduce the number of trainable parameters.

In the standard self-attention mechanism, the total number of parameters for query, key, and value transformations is:

$$P_{\text{full}} = 3 \times (d_{\text{model}} \times d_{\text{model}}) \quad (30)$$

where:

- d_{model} is the embedding dimension.
- Each of the three weight matrices (W_Q, W_K, W_V) has a size of $d_{\text{model}} \times d_{\text{model}}$.

After applying truncated SVD, each weight matrix is decomposed into three smaller matrices U , S , and V , where U and V are of reduced rank r :

$$W \approx USV^T, \quad U \in \mathbb{R}^{d_{\text{model}} \times r}, \quad S \in \mathbb{R}^{r \times r}, \quad V \in \mathbb{R}^{r \times d_{\text{model}}} \quad (31)$$

Thus, the total number of trainable parameters in the SVD-based self-attention mechanism is:

$$P_{\text{SVD}} = 3 \times (d_{\text{model}} \times r + r \times r + r \times d_{\text{model}}) \quad (32)$$

Since $r \ll d_{\text{model}}$, the reduction in parameters can be expressed as:

$$\frac{P_{\text{SVD}}}{P_{\text{full}}} = \frac{3 \times (d_{\text{model}} \times r + r^2 + r \times d_{\text{model}})}{3 \times d_{\text{model}}^2} = \frac{2rd_{\text{model}} + r^2}{d_{\text{model}}^2} \quad (33)$$

For small values of r , the parameter reduction is approximately:

$$P_{\text{SVD}} \approx \frac{2r}{d_{\text{model}}} P_{\text{full}} \quad (34)$$

which shows that using a lower rank r leads to a significant reduction in the number of parameters, especially when $r \ll d_{\text{model}}$.

4.7.3 PyTorch Implementation

```
class CustomSelfAttention(nn.Module):
    def __init__(self, embed_size, heads, lowRank):
        super().__init__()
        self.embed_size = embed_size
        self.heads = heads
        self.lowRank = lowRank
        self.head_dim = embed_size // heads

        # Initialize weights
        W_q_full = nn.Linear(embed_size, embed_size, bias=False)
        W_k_full = nn.Linear(embed_size, embed_size, bias=False)
        W_v_full = nn.Linear(embed_size, embed_size, bias=False)

        # Freeze original weights
        for param in W_q_full.parameters():
            param.requires_grad = False
        for param in W_k_full.parameters():
            param.requires_grad = False
        for param in W_v_full.parameters():
            param.requires_grad = False

        # Perform SVD on weights
        with torch.no_grad():
            U_q, S_q, V_q = linalg.svd(W_q_full.weight,
                                         full_matrices=False)
            U_k, S_k, V_k = linalg.svd(W_k_full.weight,
                                         full_matrices=False)
```

```

        U_v, S_v, V_v = linalg.svd(W_v_full.weight,
                                   full_matrices=False)

# Truncated SVD
U_q, S_q, V_q = U_q[:, :lowRank], torch.diag(S_q[:lowRank
    ]), V_q[:lowRank, :]
U_k, S_k, V_k = U_k[:, :lowRank], torch.diag(S_k[:lowRank
    ]), V_k[:lowRank, :]
U_v, S_v, V_v = U_v[:, :lowRank], torch.diag(S_v[:lowRank
    ]), V_v[:lowRank, :]

# Convert SVD components to trainable parameters
self.U_q = nn.Parameter(U_q)
self.S_q = nn.Parameter(S_q)
self.V_q = nn.Parameter(V_q)

self.U_k = nn.Parameter(U_k)
self.S_k = nn.Parameter(S_k)
self.V_k = nn.Parameter(V_k)

self.U_v = nn.Parameter(U_v)
self.S_v = nn.Parameter(S_v)
self.V_v = nn.Parameter(V_v)

def forward(self, value, key, query, mask=None):
    batch_size = query.shape[0]

# Reconstruct weights using SVD components
W_q = self.U_q @ self.S_q @ self.V_q
W_k = self.U_k @ self.S_k @ self.V_k
W_v = self.U_v @ self.S_v @ self.V_v

# Compute Queries, Keys, and Values
Q = torch.matmul(query, W_q.T).view(batch_size, -1, self.
    heads, self.head_dim).transpose(1, 2)
K = torch.matmul(key, W_k.T).view(batch_size, -1, self.
    heads, self.head_dim).transpose(1, 2)
V = torch.matmul(value, W_v.T).view(batch_size, -1, self.
    heads, self.head_dim).transpose(1, 2)

# Compute attention scores
attention_scores = torch.matmul(Q, K.transpose(-2, -1)) /
    math.sqrt(self.head_dim)
if mask is not None:
    attention_scores = attention_scores.masked_fill(mask
        == 0, float("-inf"))

attention_probs = torch.nn.functional.softmax(
    attention_scores, dim=-1)
output = torch.matmul(attention_probs, V)

```

```
# Reshape output back to original shape
output = output.transpose(1, 2).contiguous().view(
    batch_size, -1, self.embed_size)
return output
```

4.7.4 Analysis

- **Parameter Reduction:** By applying truncated SVD, the number of trainable parameters is significantly reduced.
- **Computational Efficiency:** Since lower-rank matrices are used, the computational complexity of matrix multiplications is lower.
- **Preserving Attention Mechanism:** Despite reducing the number of parameters, the model retains its ability to capture important features.

4.8 Analysis of Plots and Performance Results

To evaluate the impact of rank reduction in the Transformer model using SVD, we analyze its accuracy, training time, and parameter efficiency. The following plots and table summarize the effect of different low-rank values on model performance.

4.8.1 Performance Summary

Table 5 provides a detailed summary of the model’s accuracy, training time, memory usage, and number of parameters for different low-rank values.

LowRank	Accuracy (%)	Training Time (s)	Memory Usage (MB)	Num Parameters	Accuracy/Time
1	59.072	230.72	2884.0	4,040,069	0.256
5	55.984	231.27	2340.0	4,043,213	0.242
10	56.584	231.02	2340.0	4,047,278	0.244
15	63.456	232.47	2338.0	4,051,493	0.272
20	61.288	231.55	2272.0	4,055,858	0.264
35	60.140	232.19	2338.0	4,069,853	0.259
50	67.500	232.36	2338.0	4,085,198	0.290
70	72.596	232.27	2304.0	4,107,758	0.312
100	71.396	232.09	2338.0	4,146,098	0.307

Table 5: Performance Summary of Low-Rank Transformer Model

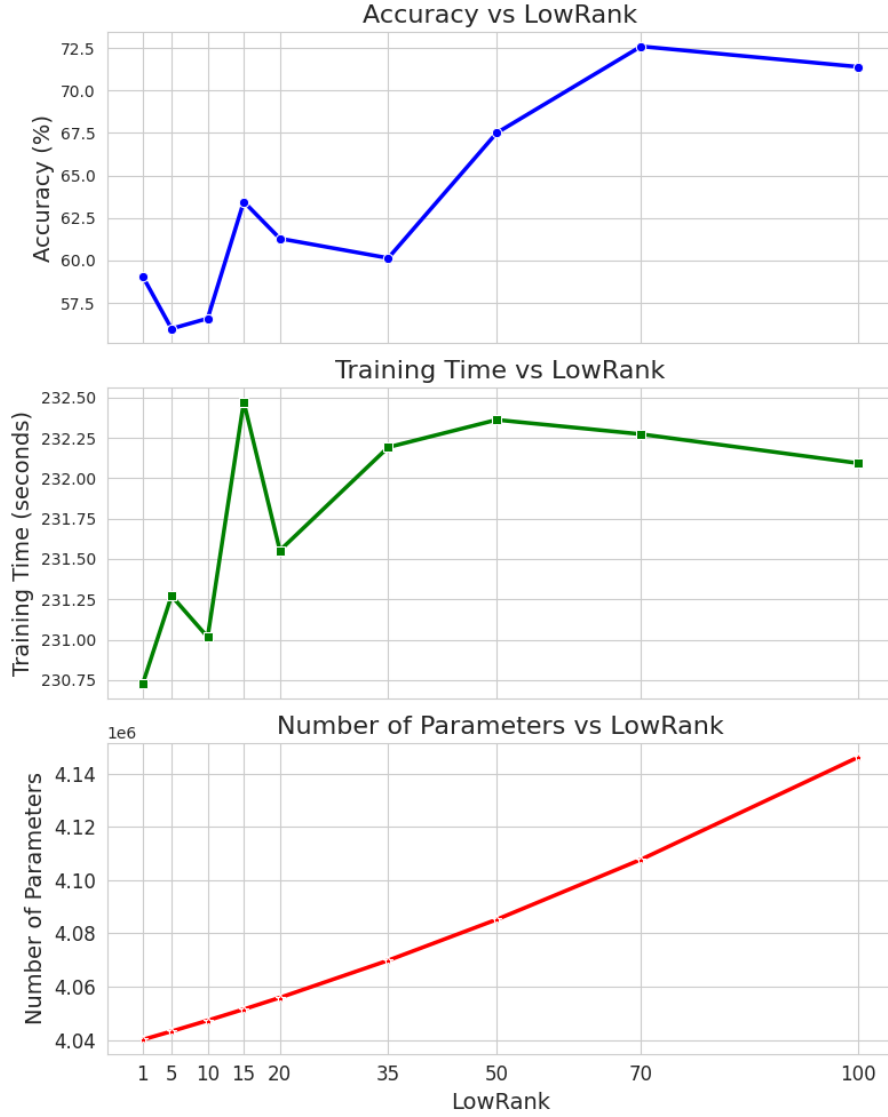


Figure 20: Impact of Low-Rank on Accuracy, Training Time, and Number of Parameters

- Increasing the rank initially improves accuracy, but the benefit diminishes at higher ranks.
- The number of parameters increases with rank, as expected.

4.9 Comparison of Standard Transformer and SVD-Optimized Transformer

To assess the effectiveness of applying SVD to the Transformer model, we compare the performance of the standard Transformer and the Low-Rank Transformer across key metrics, including accuracy, training time, memory usage, and the number of parameters.

4.9.1 Performance Comparison

Table 6 summarizes the key performance differences between the two models.

Model	Test Accuracy (%)	Training Time (s)	Number of Parameters
Standard Transformer	82.73	112.20	4,302,594
Low-Rank Transformer (Best Rank)	72.60	232.27	4,107,758

Table 6: Comparison of Standard Transformer and Low-Rank Transformer

4.9.2 Key Observations

The following points summarize the effects of applying SVD to the Transformer model:

- **Reduction in Parameters:** The Low-Rank Transformer achieves a **4.5% reduction** in trainable parameters compared to the standard model.
- **Impact on Accuracy:** The accuracy of the Low-Rank Transformer is lower than the standard model, dropping from **82.73%** to **72.60%**. This trade-off is expected due to the rank-based approximation.
- **Increase in Training Time:** The Low-Rank Transformer requires **more training time** per epoch, with a total training time of **232.27 seconds**, compared to **112.20 seconds** for the standard Transformer. The added matrix multiplications in the SVD-based model contribute to this increase.
- **Computational Cost vs. Model Size:** While parameter reduction helps decrease memory usage, the added complexity of reconstructing weight matrices from SVD introduces a computational overhead that affects training time.

4.9.3 Analysis of SVD Effectiveness in Transformer Models

While SVD is an effective method for reducing the number of trainable parameters in deep learning models, its impact in our Transformer implementation appears less significant than expected. The primary reason for this lies in the proportion of parameters allocated to different components of the model.

Impact of Embedding Parameters A major portion of the total parameters in a Transformer model comes from the embedding layers, which remain unchanged by the SVD-based decomposition. Since embedding layers in NLP models contain large lookup tables for word representations, the relative effect of parameter reduction in fully connected layers is diminished.

Effect of Small Weight Matrices In our implementation, the weight matrices of self-attention and feedforward layers are relatively small compared to the embeddings. Since SVD applies rank-based approximation to weight matrices, its overall parameter reduction effect is not as substantial in our model. This results in only a modest decrease in the number of parameters, making the trade-off between accuracy and efficiency less favorable.

Scalability to Large Models Despite the limited impact in this setting, SVD remains a powerful tool for parameter reduction in large-scale Transformer models, such as BERT or GPT-style architectures. In these models, weight matrices are significantly larger, and reducing their rank can lead to considerable memory and computational savings.

Conclusion The limited parameter reduction observed in our Transformer model suggests that **SVD is more effective in large-scale architectures** where the proportion of weight matrix parameters is higher. Future optimizations could explore **low-rank approximations for embedding layers** or hybrid compression techniques to achieve a better balance between efficiency and performance.

5 Conclusion

This report highlights the effectiveness of SVD and LoRA in reducing computational complexity while maintaining model accuracy. Further research could explore hybrid approaches combining multiple decomposition techniques.